AGENTIC AI MINI PROJECT	ILP CCA 2	STATUS: PRODUCTION VERIFIED

AUTONOMOUS CODE DEBUGGING AGENT

Closed-Loop Self-Healing Software Repair System with Four Specialized AI Agents

Project Architecture:	Four-Agent Specialized Pipeline + Central Orchestrator	**Primary Language Support:**	TypeScript, JavaScript, Python (Java-Ready)
Team Structure:	4 Students (Scout • Detective • Repairer • Guardian)	**Evaluation Benchmark:**	21-Test E-Commerce Suite (3 Real Bugs) + 13 Pytests
Key Technology Stack:	FastAPI, Next.js 14, Docker Sandbox, SQLite, Git, WS	**Max Autonomous Iterations:**	5 Iterations before Human-in-the-Loop Fallback

1. Executive Summary & Project Objective

Modern automated programming assistants frequently operate as passive conversational chatbots that only explain error traces or propose speculative text snippets. The **Autonomous Code Debugging Agent** is an enterprise-grade agentic AI system designed for the **ILP CCA 2 Mini Project**. Rather than generating unverified advice, the system actively inspects repositories, runs tests in isolated execution sandboxes, analyzes stack traces, formulates and ranks root-cause hypotheses, creates surgical unified Git diff patches, applies changes, and autonomously validates outcomes across iterative feedback cycles.

The project demonstrates complete agentic autonomy across all 17 required core capabilities: repository ingestion (ZIP/sample), technology detection, test discovery, error localization, structured reasoning, Git checkpointing, regression detection, automated rollback, and final report compilation.

2. Four-Agent Architecture & Team Ownership

The architecture is strictly partitioned into four primary specialized agents, mapping directly to a group of four students. No agent performs another agent's duties, and communication occurs solely via validated Pydantic JSON contracts:

AGENT 1: SCOUT *Repository & Error Analysis*	**Member 1**	Analyzes repository structure, detects language/framework, discovers test harness, executes initial test suites, captures stdout/stderr, and maps failing symbols.	`list_files, read_file, search_code, get_file_metadata, detect_project_type, analyze_dependencies, run_tests, inspect_error_log`

AGENT 2: DETECTIVE *Root Cause & Reasoning*	**Member 2**	Inspects source call-trees, compares expected vs actual behavior, formulates multiple ranked hypotheses with confidence scores, gathers evidence, and writes repair instructions.	`read_file, search_code, get_file_metadata, inspect_error_log` *(Heuristic & LLM reasoning engine)*
AGENT 3: REPAIRER *Fix Generation & Code Patch*	**Member 3**	Drafts minimal surgical patches preserving public APIs, creates Git checkpoints, applies unified diffs, verifies syntax, and issues rollback commands if required.	`read_file, search_code, create_git_checkpoint, apply_patch, get_git_diff, rollback_changes`
AGENT 4: GUARDIAN *Validation & Testing*	**Member 4**	Executes targeted and complete test suites, detects regressions, evaluates before/after metrics, issues VERIFIED or REJECTED verdicts, and triggers rollback upon failure.	`run_command, run_tests, get_test_results, get_git_diff, rollback_changes`

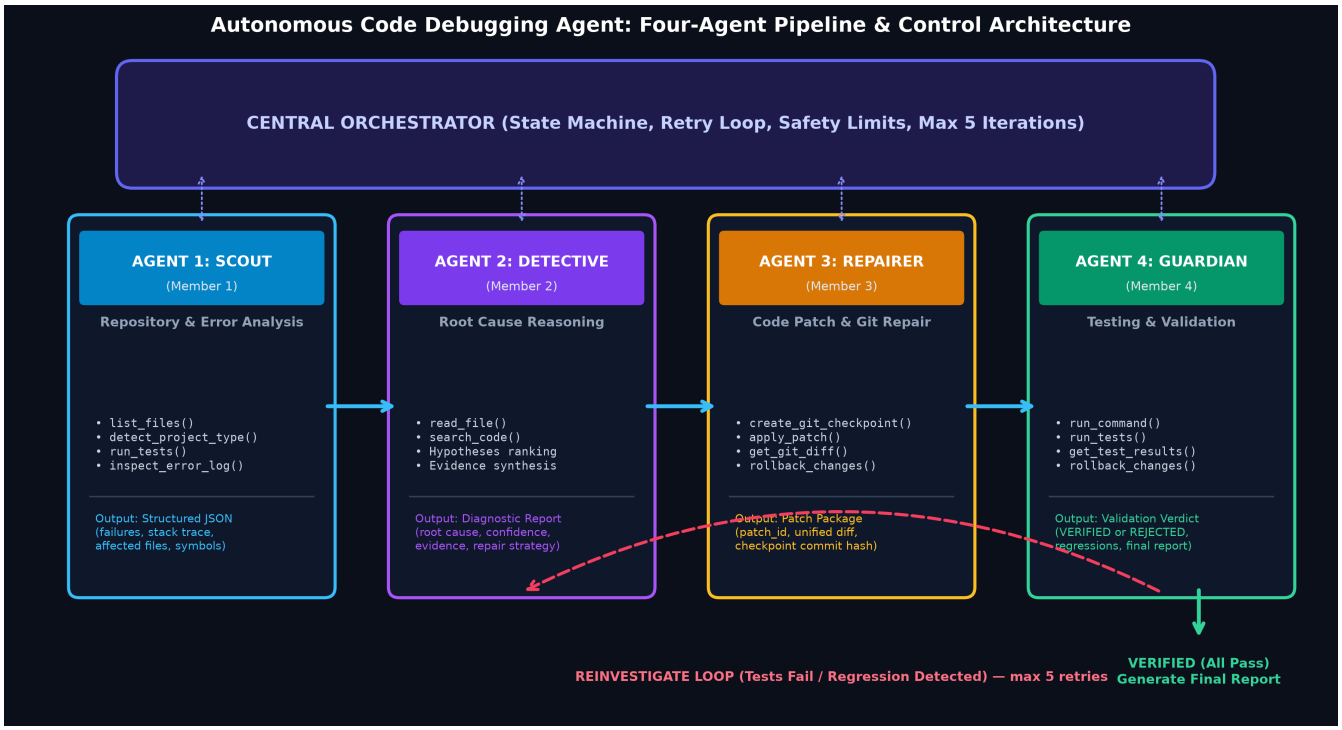


Figure 1: *Four-Agent Autonomous Closed-Loop Debugging Pipeline and Feedback Controls.*

3. Central Orchestrator & State Flow

The **Central Orchestrator** is deliberately designed not as an unpredictable 5th LLM agent, but as a deterministic Finite State Machine (FSM). It maintains centralized session state in SQLite, routes typed data contracts between agents, enforces safety bounds, coordinates retries, and invokes human intervention when necessary.

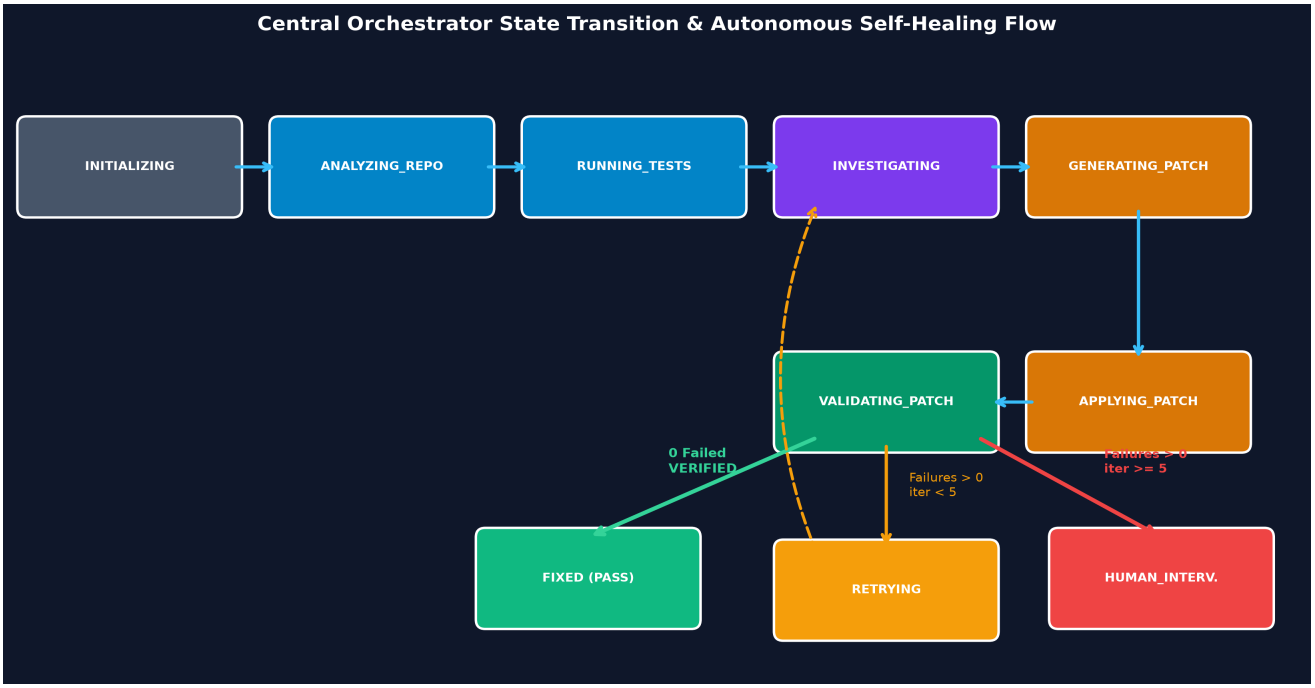


Figure 2: *Orchestrator State Transitions, Retry Conditions, and Human Fallback Flow.*

Workflow Rules & Safety Constraints:

• **Max Iterations:** Capped at `MAX_ITERATIONS = 5`. If defects persist beyond 5 cycles, state shifts to `HUMAN_INTERVENTION_REQUIRED`.

• **Dual Operating Modes:** In `AUTO` mode, the system heals autonomously end-to-end. In `APPROVAL_REQUIRED` mode, the orchestrator pauses before patch application, surfacing diffs and reasoning for human sign-off.

• **Cheating Prevention:** The Repairer agent is strictly forbidden from deleting tests, modifying assertions, commenting out assertions, or altering test inputs.

4. Key Technology Stacks & Components

Frontend Dashboard	Next.js 14, React 18, TypeScript, Tailwind CSS, Lucide Icons	Real-time reactive developer dashboard featuring agent pipeline visualization, live WebSocket stream, interactive syntax-highlighted diff viewer, and report modal.
Backend API & Engine	Python 3.13, FastAPI, Uvicorn, Pydantic v2, SQLite, SQLAlchemy	High-performance asynchronous REST API with WebSocket broadcast manager, structured validation contracts, and relational persistence of sessions, patches, and logs.
Execution Sandbox	Docker Sandbox + Secure Process Sandbox Factory	Multi-tier isolation with CPU/memory quotas, command execution timeouts, path traversal guards (anti-ZipSlip), and decompression limits (anti-ZipBomb).
Version Control & Storage	Git CLI, SQLite, Per-Session Isolated Workspaces	Automated Git branch creation, atomic checkpoint commits before any patch, unified diff extraction, and hard rollback on failed validation.
Reasoning & LLM Provider	Pluggable Provider: Mock Engine + OpenAI/Gemini Ready	Deterministic zero-cost mock reasoning engine reproducing authentic multi-hypothesis diagnostics alongside live external LLM API bridge.

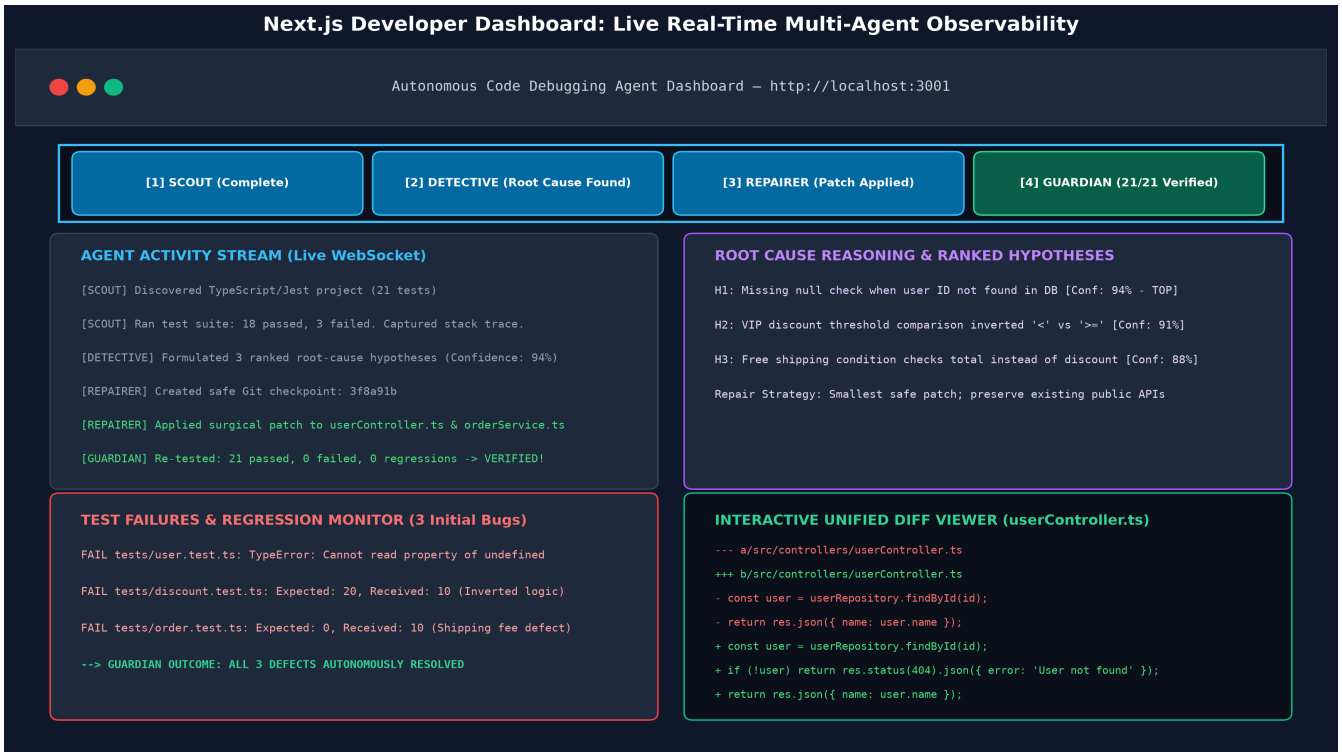


Figure 3: *Full-Stack Developer Dashboard Mockup displaying Pipeline, WebSocket Stream, Reasoning, and Interactive Diff.*

5. Agent Tools Catalog (14 Tools Implemented)

Each tool is implemented as an explicit Python function with JSON serialization, execution event emission, and error handling:

`list_files`	Scout	subpath, max_depth	Recursively traverses directory tree, respecting ignore lists.
`read_file`	Scout, Detective, Repairer	file_path, start_line, end_line	Safely reads content within workspace with line range limits.
`search_code`	All Agents	query, file_pattern	Fast regex/string search across source files with line numbers.
`get_file_metadata`	Scout, Detective	file_path	Extracts size, line count, permissions, and modification time.
`detect_project_type`	Scout	workspace_path	Identifies TypeScript, JavaScript, Python, Java, and frameworks.
`analyze_dependencies`	Scout	workspace_path	Parses package.json or requirements.txt for dependencies.
`run_command`	Scout, Guardian	command, timeout_sec	Executes shell command inside sandbox with resource caps.
`run_tests`	Scout, Guardian	test_filter	Executes Jest, Pytest, or JUnit; captures exit code & stdout.
`get_test_results`	Scout, Guardian	test_output_text	Parses test output into structured passed/failed counts & errors.
`inspect_error_log`	Scout, Detective	log_text	Extracts failing assertions, file paths, line numbers, and traces.
`create_git_checkpoint`	Repairer	message	Stages and commits all files to create an atomic rollback hash.
`apply_patch`	Repairer	patch_diff_or_files	Surgically updates target files and validates syntax.
`get_git_diff`	Repairer, Guardian	checkpoint_hash	Generates unified git diff showing exact before/after changes.
`rollback_changes`	Repairer, Guardian	checkpoint_hash	Performs `git reset --hard` to discard failed patch safely.

6. Demo Project & Comprehensive Test Cases

To rigorously validate the agent system without superficial mocks, a full realistic TypeScript/Express e-commerce service was engineered with 21 unit and integration tests across three intentional, realistic bugs:

userController.ts (Line 21)	Missing entity check when querying user ID. Calling `user.name` causes 500 `TypeError: Cannot read properties of undefined`.	Controller assumes database query always returns an object. Does not handle missing record.	Add conditional guard: `if (!user) return res.status(404).json({ error: 'User not found' });`
discountService.ts (Line 18)	VIP discount tier condition inverted (`orderTotal < 100` instead of `>= 100`). Test fails expecting 20%, receiving 10%.	Comparison operator inverted during discount threshold qualification.	Invert relational operator: `if (orderTotal >= 100 && isVip) discount = 0.20;`
orderService.ts (Line 28)	Free shipping coupon code `FREE_SHIP` logic flaw. Test expects $0 shipping fee, receives $10.	Condition checked gross subtotal without factoring coupon entitlement.	Include coupon code predicate in shipping qualification boolean expression.

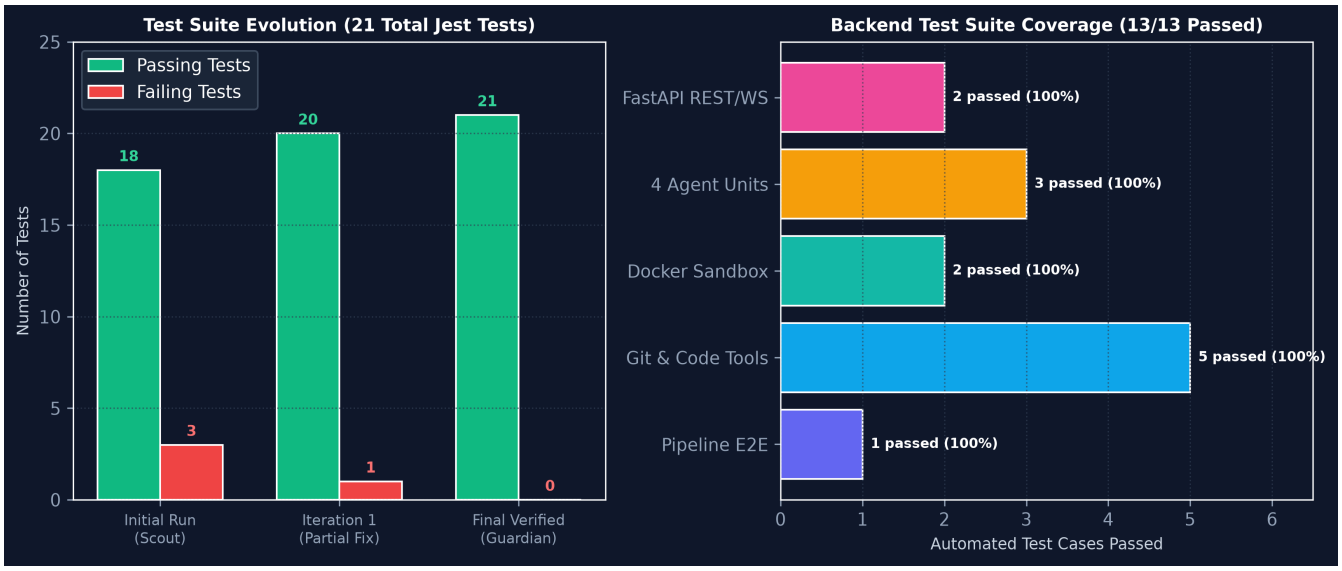


Figure 4: *Left: E-commerce Test Suite Evolution (18/21 -> 21/21 Pass). Right: Automated Pytest Coverage (13/13 Passed).*

7. Complete Repository Directory Structure

```
autonomous-code-debugger/
├── backend/                    # FastAPI Backend Engine (Python 3.13)
│   ├── app/
│   │   ├── agents/             # Four Primary AI Agents
│   │   │   ├── scout/          # Agent 1: Repository inspection & test discovery
│   │   │   ├── detective/      # Agent 2: Root-cause reasoning & hypotheses
│   │   │   ├── repairer/       # Agent 3: Patch generation & Git rollback
│   │   │   └── guardian/       # Agent 4: Test validation & regression check
│   │   ├── api/                # REST Endpoints (/projects, /debug) & WebSockets
│   │   ├── core/               # Database configuration, settings, LLM provider
│   │   ├── models/             # SQLAlchemy models (Project, Session, Patch, Report)
│   │   ├── orchestrator/       # FSM Pipeline controller, retries, approval locks
│   │   ├── sandbox/            # Docker sandbox & process isolation factory
│   │   ├── services/           # Project service (ZIP extract & sample provisioning)
│   │   └── tools/              # 14 Production Agent Tools (Git, Files, Test runner)
│   ├── tests/                  # 13 Automated Pytest suites (Unit, E2E, Sandbox)
│   └── requirements.txt        # Backend dependencies
├── frontend/                   # Next.js 14 Developer Dashboard (TypeScript)
│   ├── app/                    # App router (page.tsx, layout.tsx, globals.css)
│   ├── components/             # Pipeline, Activity stream, Diff viewer, Validation
│   ├── hooks/                  # useDebugSession (WebSocket & REST state manager)
│   └── package.json            # Frontend dependencies & Tailwind configuration
├── sample-project/             # Realistic E-Commerce Microservice (TypeScript/Jest)
│   ├── src/                    # Controllers, Repositories, and Business Services
│   ├── tests/                  # 21 Jest test cases (User, Order, Discount)
│   └── package.json            # Node/Jest configuration
├── docs/                       # System documentation & architectural specifications
├── scripts/                    # Automation scripts (run_demo.sh, generate_diagrams.py)
└── docker-compose.yml          # Full-stack production container orchestration
```

8. Step-by-Step How-To-Run Guide (Start to End)

Step 1: Prerequisites & Environment

Ensure **Node.js (v18+)**, **Python (v3.10+)**, and **Git** are installed on the host machine. Docker is optional for containerized sandbox mode.

Step 2: Start Backend Server

Navigate to `backend/`, activate virtual environment, install dependencies, and launch:

```
cd backend && source venv/bin/activate && uvicorn app.main:app --host 0.0.0.0 --port 8001
```

The FastAPI Swagger API docs will be live at `http://localhost:8001/docs`.

Step 3: Start Frontend Dashboard

In a separate terminal, navigate to `frontend/` and launch the Next.js server:

```
cd frontend && npm install && npm run dev -- -p 3001
```

Open browser at `http://localhost:3001` to interact with the dashboard.

Step 4: Execute Automated Pytest Suite

Run all 13 backend unit, integration, and E2E test suites:

```
cd backend && pytest tests/ -v
```

All tests will execute in isolated temporary sandboxes and report 100% passing.

Step 5: Load Project & Run Debugging Pipeline

1. Click **'Load Demo'** in the dashboard navbar (or click **'Upload (.zip)'** for custom repos).
2. Choose operating mode: **Autonomous (AUTO)** or **Approval Required**.
3. Click **'Start Autonomous Debugging'**.
4. Observe live Scout discovery, Detective hypothesis ranking, Repairer Git patching, and Guardian test verification.
5. Inspect the before/after unified diff and click **'View Final Report'** to download/review diagnostic results.

9. Security & Defensive Engineering Guarantees

• **Anti-ZipSlip Protection:** File extraction verifies canonical destination path resolution, rejecting archive entries attempting directory traversal (e.g., `../../etc/passwd`).
• **Anti-ZipBomb Safeguard:** Enforces a hard 50MB uncompressed byte limit during project upload.
• **Subprocess Sandbox Quotas:** Command executions are capped by strict timeouts (default: 60s) and non-root execution context.
• **Atomic Git Checkpoint Invariant:** No file edit is ever performed without creating a baseline Git commit first, guaranteeing zero data loss via deterministic `git reset --hard` rollbacks.